Lesson 4 — Polling and Interrupt Handling

ATmega328P / Arduino Uno | Source: lecture 04 slides, Exercise 3, peer notes | Builds on Lesson 3 (DDR, PORT, PIN, pull-ups)

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Compare polling and interrupts, and explain the two problems of polling.
- Explain the 3 steps of interrupt handling (vector table, ISR, stack).
- Set up an **external interrupt** (INT0 / INT1) and a **pin change interrupt** (PCINT) in C and Arduino.
- Explain switch bounce and how a capacitor debounces it.
- Design the Exercise 3 "click counter".

GIVEN TO YOU IN THE EXAM (DO NOT MEMORISE THESE)

ATmega328P Simplified Data Sheet: the interrupt vector names (`INT0_vect` , `PCINT2_vect` …), the bits of `EICRA` , `EIMSK` , `PCICR` , `PCMSK0/1/2` , and the ISC table (low level / logic change / falling / rising). **AVR C cheat sheet**: `#include <avr/interrupt.h>` , `sei()` , `cli()` , `ISR(vector){ }` , `volatile` . **Arduino cheat sheet**: `attachInterrupt` , `digitalPinToInterrupt` , `LOW / CHANGE / RISING / FALLING` , `interrupts()` / `noInterrupts()` ; pins 2 and 3 are marked INT0 and INT1 on the board picture.

Not given, so learn them here: polling vs interrupt, the handling steps, priority, which register does what, how to combine them in code, debouncing, and the PCINT numbering.

1. The problem: doing two things "at the same time"

The CPU runs one instruction at a time. While it is busy blinking an LED with a 2-second delay, a button might be pressed. How does the CPU notice? Two approaches:

POLLING

The CPU **keeps checking** the status of each device at regular intervals while doing its main task. If a device needs service, it serves it, then continues.

Like checking your phone every 5 minutes to see if a message came.

INTERRUPT

The **device tells the CPU** when it needs service. The CPU pauses the main task, serves the device (runs the ISR), then resumes exactly where it stopped.

Like the phone ringing: you only react when it rings.

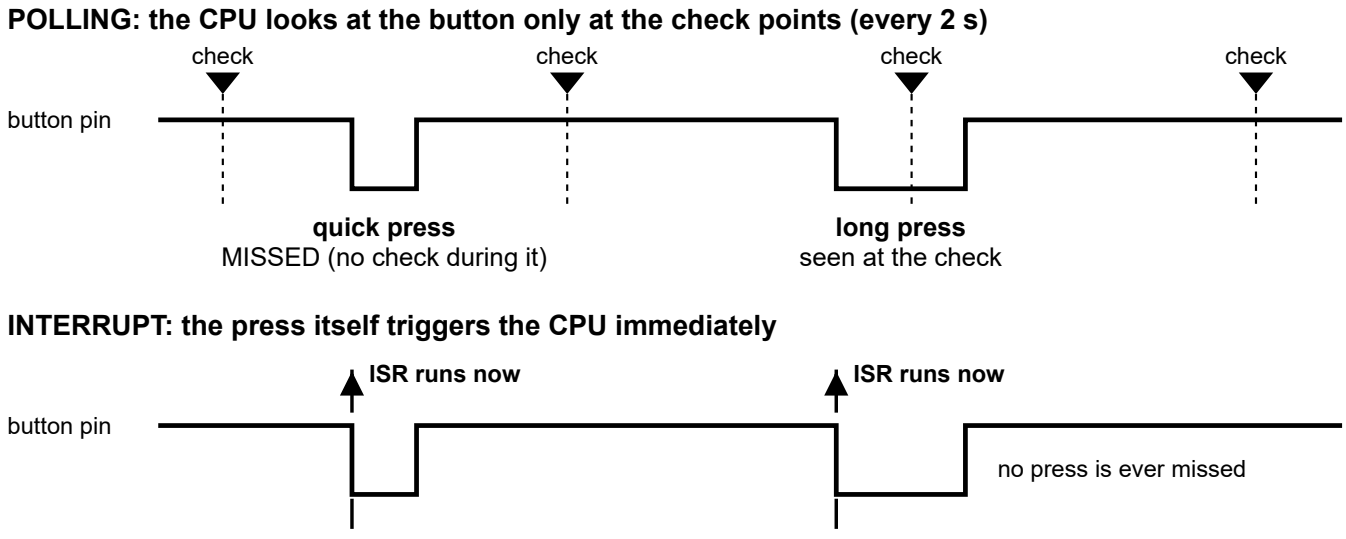


Figure 1. With polling, a press shorter than the checking interval can be missed completely.

2. System 5: `pollingBasedTrigger`

Task: blink the LED on **PB5** (pin 13) every 2 s, and toggle a second LED on **PB0** (pin 8, through 220 Ω; the on-board pin 13 LED has its own 1 kΩ) when the button on **PD2** (pin 2) is pressed. Button between PD2 and GND, internal pull-up on.

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main(void)
{
  DDRB = DDRB | 0b00100001;   // PB0, PB5 outputs
  DDRD = DDRD & 0b11111011;   // PD2 input
  PORTD = PORTD | 0b00000100; // pull-up on PD2
  while (1)
  {
    PORTB = PORTB ^ 0b00100000;     // toggle PB5
    _delay_ms(2000);                // 2 s
    if ((PIND & 0b00000100) == 0)   // PD2 low?
      PORTB = PORTB ^ 0b00000001;   // toggle PB0
  }
  return 0;
}
```

Line	Meaning
`DDRB | 0b00100001`	Bits 5 and 0 set: both LED pins are outputs.
`DDRD & 0b11111011`	Bit 2 cleared: PD2 is an input.
`PORTD | 0b00000100`	Pull-up on PD2: reads 1 when released.
`PORTB ^ 0b00100000`	**Main task:** flip the PB5 LED.
`if ((PIND & 0b00000100) == 0)`	**The poll:** keep only bit 2. Result 0 means PD2 is LOW = **pressed**.
`PORTB ^ 0b00000001`	**The service:** flip the PB0 LED.

Arduino version: `pinMode(2, INPUT_PULLUP)`, then in `loop()`: `digitalWrite(13, !digitalRead(13)); delay(2000); if (!digitalRead(2)) digitalWrite(8, !digitalRead(8));` `!digitalRead(13)` reads the pin's current state and writes the opposite, which is a toggle.

THE TWO PROBLEMS OF POLLING (LEARN BOTH)

1. **Missed events:** if the device changes state too fast, or the CPU's checking interval is too long, the change can go undetected. Here the button is checked only once every 2 s, so a quick press during the delay is lost.
2. **No priority:** the CPU checks all devices in the same fixed order every time, so an urgent device cannot be served first.

Also: the CPU wastes time checking devices that usually do not need anything.

Polling done better: detect the press, not the held button

System 5 toggles PB0 every 2 s **while** the button is held, and misses short presses. In practical tasks (counters, reaction timers) you usually want "react once per press". The standard trick is to remember the previous reading and look for the change from 1 to 0, a **falling edge**, then wait out the bounce.

```
unsigned char last = 1;              // 1 = released
while (1)
{
    unsigned char now = (PIND & (1<<PD2)) ? 1 : 0;
    if (last == 1 && now == 0)       // it was up, now down
    {
        PORTB ^= (1<<PB0);           // react once
        _delay_ms(30);               // ignore the bounce
    }
    last = now;                      // remember for next time
}
```

```
int last = HIGH;                     // Arduino version
void loop()
{
  int now = digitalRead(2);
  if (last == HIGH && now == LOW)    // falling edge
  {
    digitalWrite(8, !digitalRead(8));
    delay(30);                       // software debounce
  }
  last = now;
}
```

The loop runs thousands of times per second (no long delay), so no press is missed. This is **software debouncing**; the 100 nF capacitor is **hardware debouncing**. Either is acceptable; with interrupts, use the capacitor (you cannot `delay` inside an ISR).

3. Interrupts

A peripheral device **interrupts** the CPU when it needs service. The CPU **suspends** its current task, runs the **Interrupt Service Routine (ISR)** for that device, then **resumes** the task. This frees the CPU from polling and **supports priority**.

The 3 steps of interrupt handling

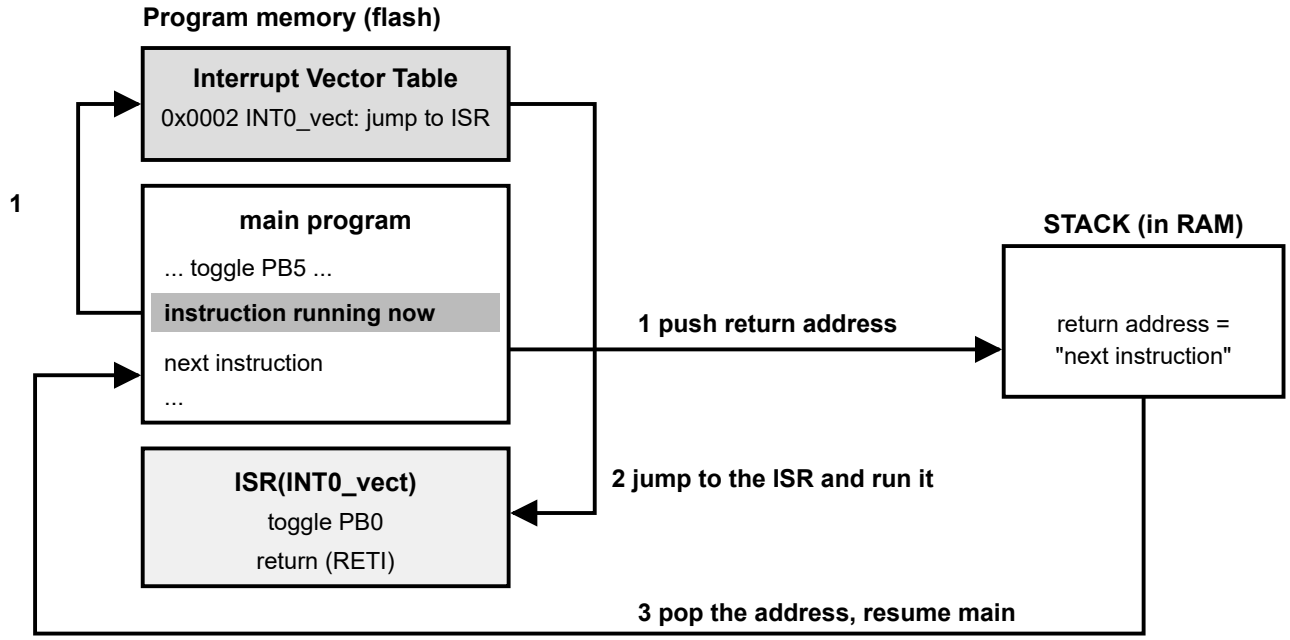


Figure 2. (1) The interrupt arrives: the CPU finishes the current instruction, saves the return address on the stack and jumps to the vector-table entry. (2) The entry jumps to the ISR, which runs. (3) At the end of the ISR the address is popped from the stack and main continues where it stopped.

Interrupt Vector Table

A table at the start of flash. Each entry holds the address (a jump) of one ISR. **Lower address = higher priority.** If two interrupts arrive together, the lower-address one runs first.

Address	Vector name
0x0002	INT0_vect (highest of these)
0x0004	INT1_vect
0x0006	PCINT0_vect (Port B)
0x0008	PCINT1_vect (Port C)
0x000A	PCINT2_vect (Port D)
0x0012	TIMER2_OVF_vect
0x001A	TIMER1_OVF_vect
0x0020	TIMER0_OVF_vect
0x0024	USART_RX_vect
0x0028	USART_TX_vect
0x002A	ADC_vect (lowest of these)

The names are on your exam data sheet; the addresses (and so the priority order) are not.

Writing an ISR in C

```
#include <avr/interrupt.h>

ISR(INT0_vect)        // runs on its own
{                     // when INT0 fires
    PORTB ^= (1 << 0);
}                     // never call it yourself
```

- `sei()` = SEt global Interrupt enable. `cli()` = CLear it (turn all interrupts off).
- Keep an ISR **short**: no long delays inside it, or the main task and other interrupts are blocked.
- A variable shared between an ISR and `main` must be **volatile**, so the compiler always re-reads it from RAM instead of keeping an old copy.

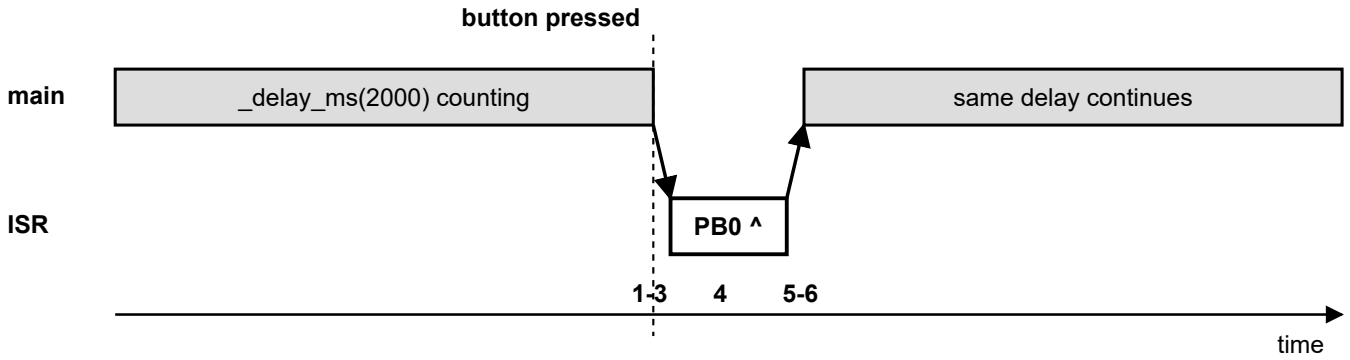
The flag pattern (from the lecture)

```
volatile int flag = 0;
int main(void) {
  ...
  while (1) {
    if (flag == 0) { /* normal work */ }
    else           { /* react, then flag = 0 */ }
  }
}
ISR(INT0_vect) { flag = 1; }   // just note it
```

3A. Interrupts from zero: what really happens

Word	Plain meaning
Peripheral	A hardware block inside the chip that works on its own beside the CPU: the I/O ports, the external-interrupt detector, timers, ADC, USART.
Event	Something a peripheral notices: a falling edge on PD2, a timer overflowing, an ADC conversion finishing, a byte arriving.
Flag	A single bit a peripheral sets to 1 to say "my event happened". E.g. `INTF0` in register `EIFR` becomes 1 on an INT0 edge. Hardware sets it, even if interrupts are off.
Enable bit	A bit **you** set to say "if that flag rises, interrupt the CPU". Without it the flag still rises but nothing happens.
Vector	A fixed slot in flash for each interrupt source. The CPU jumps to the slot; the compiler puts a jump to your ISR there.
Stack	An area of RAM used like a pile of plates: the last thing put on is the first taken off. The CPU puts the return address on it so it knows where to come back to.
ISR	Interrupt Service Routine: a function you write with `ISR(vector_name)`. You never call it; the hardware does.
Program counter (PC)	The CPU register holding the address of the next instruction. Interrupt handling is just saving the PC, changing it, and restoring it.

Walk-through: System 6, button pressed in the middle of the 2 s delay



The ISR takes only a few microseconds, so the blinking LED is not visibly disturbed. The delay simply resumes.

Figure 2A. The main program is paused for the ISR and then continues from the exact instruction where it stopped.

1. **The event:** the button pulls PD2 from 5 V to 0 V. The INT0 detector sees a falling edge (EICRA says that is what we want) and sets the flag `INTF0 = 1`.
2. **The check:** after every instruction the CPU asks "is any flag set whose enable bit is 1, and is the global I bit 1?" INTF0 = 1, INT0 enable = 1, I = 1: yes.
3. **Save and block:** the CPU finishes the current instruction, pushes the PC (return address) onto the stack, and **clears the I bit** so no other interrupt can break into this one.
4. **Jump:** the PC is loaded with INT0's vector address (0x0002); the jump there leads to `ISR(INT0_vect)`. The hardware also clears INTF0 automatically. Your code toggles PB0.
5. **Return:** the ISR ends with the `RETI` instruction (the compiler adds it). RETI pops the PC from the stack and **sets the I bit again**.
6. **Resume:** the CPU carries on inside `_delay_ms` as if nothing happened. The 2 s delay ends a few microseconds late.

Who does what	
You write	**The hardware does by itself**
Pin as input + pull-up	Watches the pin for the edge
EICRA (which edge)	Sets the flag INTF0
EIMSK (enable INT0)	Checks flags after each instruction
`sei()`	Saves the PC, clears I, jumps to the vector
The ISR body	Clears the flag, RETI restores PC and sets I

INTERRUPT LATENCY
The time from event to the first ISR instruction: the CPU finishes the current instruction, then needs about 4 cycles to save and jump, plus the jump in the vector. At 16 MHz that is well under 1 µs, so an interrupt is effectively instant.

Questions a beginner always asks

Question	Answer
What if the button is pressed during `_delay_ms`?	That is the whole point: the delay is paused, the ISR runs, the delay continues. With polling the press would be lost.
Can an ISR be interrupted by another interrupt?	Not normally: the I bit is cleared on entry. A second event sets its flag and waits; it runs right after the first ISR returns.
Two interrupts happen at the same moment?	The one with the lower vector address (higher priority) runs first, then the other.
The button is pressed 3 times while an ISR is running?	The flag is only one bit, so it can only remember "at least once". One extra run happens, the other presses are lost. Hence: keep ISRs short.
Why do I never call the ISR?	It is not called by your code; its address sits in the vector table and the CPU jumps there. Calling it yourself would end with RETI in the wrong place.
What if I enable an interrupt but write no ISR for it?	The vector points to a default handler that **resets the chip**. The program seems to restart every time the event happens. Always write the ISR for every enabled interrupt, with the exact vector name.
What happens with "low level" trigger while the button is held?	The interrupt fires again and again as long as the pin is low, so the main program barely runs. Use falling edge for buttons.
Why `volatile`?	The compiler may keep a variable in a CPU register inside a loop and never re-read it from RAM. If the ISR changes it in RAM, main would not notice. `volatile` forces a fresh read every time.

4. Two switches in a row: local enable and global enable

An interrupt reaches the CPU only if **both** its own enable bit is 1 **and** the global I bit in `SREG` is 1 (set by `sei()`). The device sets a flag when its event happens; the enables decide if that flag interrupts the CPU.

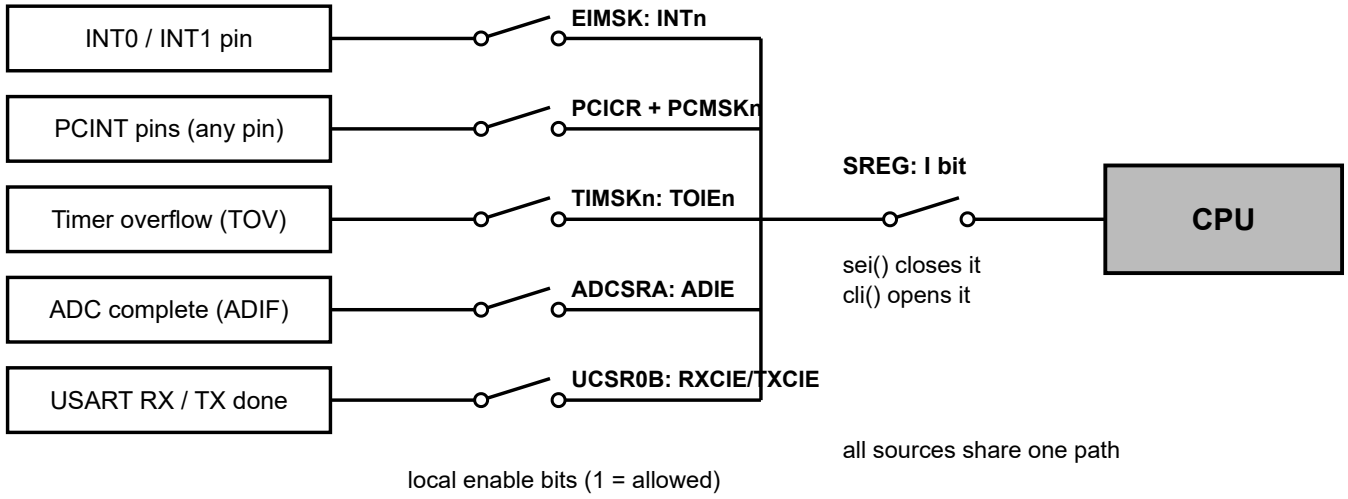
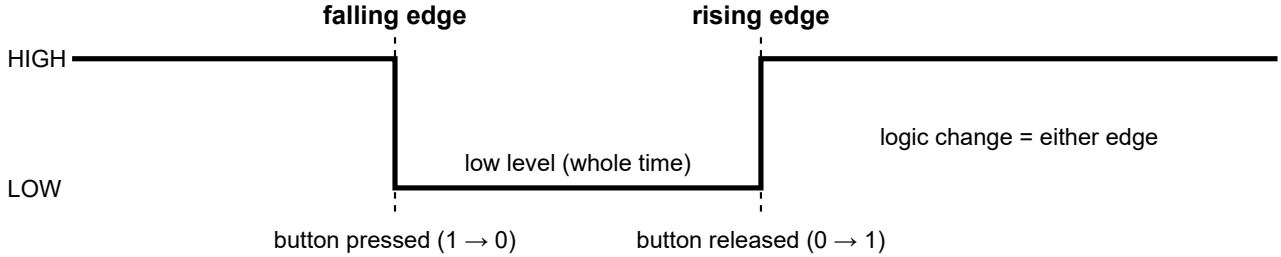


Figure 3. Each source has its own enable switch, then all go through the one global switch (I bit). Forgetting `sei()` = nothing ever interrupts.

5. External interrupts: INT0 and INT1

Two special pins: **INT0 = PD2 (Arduino pin 2)** and **INT1 = PD3 (Arduino pin 3)**. Each has its own vector and you can choose **what kind of signal** triggers it.

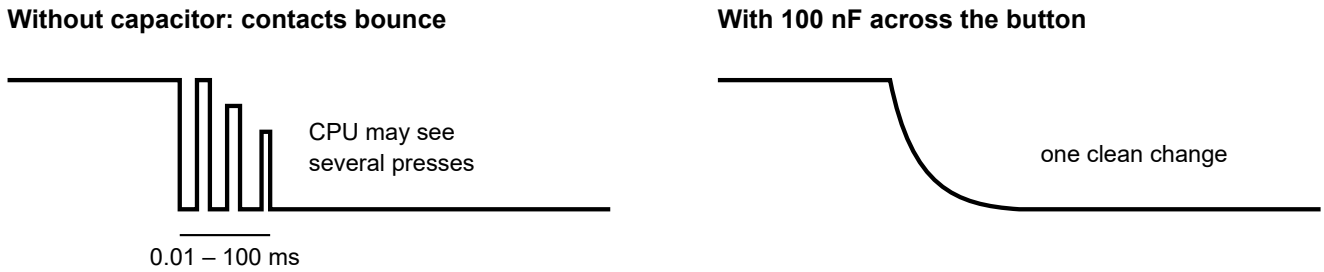


With a pull-up and a button to GND, "falling edge" means "the moment it is pressed".

Figure 4. The four trigger choices, set with the ISC bits in EICRA (table on your exam sheet).

Register	What it does	For INT0, falling edge
EICRA (External Interrupt Control Register A)	Bits ISC01/ISC00 choose the trigger for INT0; ISC11/ISC10 for INT1. 00 = low level, 01 = any change, 10 = falling, 11 = rising.	ISC01 = 1, ISC00 = 0
EIMSK (External Interrupt MaSK)	Bit INT0 / INT1 = 1 enables that interrupt.	INT0 = 1
SREG I bit	Global enable.	`sei();`

Debouncing a button



The capacitor cannot change voltage instantly, so it smooths the rapid bounces into one slow transition.

Figure 5. A mechanical switch bounces for a few ms. An interrupt is fast enough to catch every bounce, so debouncing matters here.

Why the capacitor works: the pull-up resistor (20–50 kΩ inside the chip) has to charge the 100 nF capacitor. The time constant τ = R × C ≈ 35 kΩ × 100 nF ≈ 3.5 ms, so the pin voltage cannot jump up and down faster than a few milliseconds. The bounces (each far shorter) are smoothed away and the pin crosses the HIGH/LOW threshold only once. Pressing the button discharges the capacitor almost instantly, so the press itself is still detected at once.

6. System 6: `externalInterruptBasedTrigger`

Same task as System 5, but the button press is caught by **INT0** (PD2) with a **100 nF** capacitor from PD2 to GND for debouncing. The main loop only blinks PB5; the ISR toggles PB0. No press is ever missed.

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRB = DDRB | 0b00100001;                   // PB5, PB0 outputs
    DDRD = DDRD & 0b11111011;                   // PD2 (INT0) input
    PORTD = PORTD | 0b00000100;                 // pull-up on PD2
    EICRA = (EICRA & 0b11111110) | 0b00000010;  // INT0 on falling edge
    EIMSK = EIMSK | 0b00000001;                 // enable INT0
    sei();                                      // enable global interrupts
    while (1)
    {
        PORTB = PORTB ^ 0b00100000;             // toggle PB5
        _delay_ms(2000);
    }
    return 0;
}

ISR(INT0_vect)                                  // runs on every press
{
    PORTB = PORTB ^ 0b00000001;                 // toggle PB0
}
```

Line	Meaning
`#include <avr/interrupt.h>`	Gives `sei()`, `cli()` and `ISR()`.
`EICRA = (EICRA & 0b11111110) | 0b00000010`	Two steps: `& 0b11111110` clears ISC00 (bit 0), then `| 0b00000010` sets ISC01 (bit 1). ISC01 ISC00 = **10 = falling edge**. The other bits are untouched.
`EIMSK | 0b00000001`	Local enable for INT0.
`sei()`	Global enable. Must come **after** the setup.
`while (1) { ... }`	The main task knows nothing about the button any more.
`ISR(INT0_vect)`	The CPU jumps here by itself at each falling edge on PD2.

Same with labelled constants: `EICRA = (EICRA & ~(1<<ISC00)) | (1<<ISC01); EIMSK |= (1<<INT0);`

Arduino version

```
const int ledPin1 = 8, ledPin2 = 13, switchPin = 2;
void setup()
{
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(switchPin, INPUT_PULLUP);
  int intNum = digitalPinToInterrupt(switchPin);   // pin 2 -> interrupt 0
  attachInterrupt(intNum, toggle, FALLING);        // call toggle() on a fall
}
void loop()
{
  digitalWrite(ledPin2, !digitalRead(ledPin2));    // blink pin 13
  delay(2000);
}
void toggle()                                      // the ISR
{
  digitalWrite(ledPin1, !digitalRead(ledPin1));
}
```

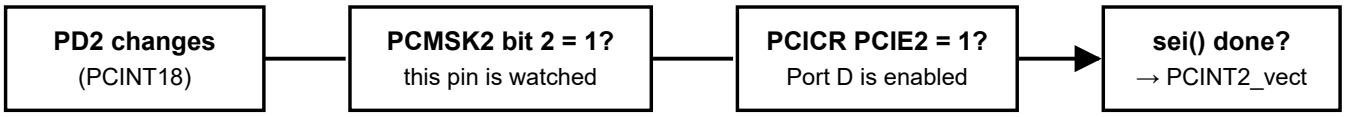
`attachInterrupt` does the EICRA, EIMSK and `sei()` work for you. Only pins **2 and 3** work on an Uno.

7. Pin change interrupts (PCINT): any pin can interrupt

Only 2 pins have INT0/INT1, but **every** I/O pin has a **pin change** interrupt. Differences: they trigger on **any change** (both edges, no choice), and all pins of one port **share one vector**, so the ISR must read the pin to find out what happened.

Port	PCINT numbers	Port enable bit in PCICR	Pin mask register	Vector
B (PB0–PB7)	PCINT0 – PCINT7	PCIE0	PCMSK0	PCINT0_vect
C (PC0–PC6)	PCINT8 – PCINT14	PCIE1	PCMSK1	PCINT1_vect
D (PD0–PD7)	PCINT16 – PCINT23	PCIE2	PCMSK2	PCINT2_vect

Trick: PD2 is PCINT18 = Port D's bit 2, so its bit in PCMSK2 is bit 2. In each port, the PCMSK bit number equals the pin's bit number.



All three "yes" are needed. The ISR then checks PIND to see if it was a press (low) or a release (high).

Figure 6. The chain for a pin change interrupt on PD2.

System 7: `pinChangeInterruptBasedTrigger`

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRB  |= (1<<5) | (1<<0);      // PB5, PB0 outputs
    DDRD  &= ~(1<<2);               // PD2 (PCINT18) input
    PORTD |= (1<<2);                // pull-up on PD2
    PCICR |= (1<<2);                // PCIE2: enable pin change interrupts on Port D
    PCMSK2 |= (1<<2);               // watch PD2 (PCINT18) only
    sei();
    while (1)
    {
        PORTB ^= (1<<5);            // toggle PB5
        _delay_ms(2000);
    }
}

ISR(PCINT2_vect)                    // any change on a watched Port D pin
{
    if ((PIND & (1<<2)) == 0)       // only act if PD2 is now LOW (falling edge = press)
        PORTB ^= (1<<0);            // toggle PB0
}
```

Why the `if` in the ISR? The ISR runs on the press **and** on the release. Without the check, PB0 would toggle twice per click and appear not to change. The slides show the same program three ways: binary (`0b00000100`), hex (`0x04`) and shifts (`1<<2`); all are identical.

Which method when?

	Polling	External interrupt (INT0/1)	Pin change interrupt
Pins	any	only PD2, PD3	any
Trigger	whatever you check	low / change / falling / rising	any change only
Vector	none	one per pin	one per port (shared)
Can miss events?	yes	no	no
Priority	no	yes (high)	yes (after INT0/INT1)

8. Exercise 3 solved: the "click counter"

Spec: button 1 is pressed repeatedly to enter a number (0–255). Button 2 means "done": the number is shown in **binary on 8 LEDs**. It stays until button 2 is pressed again, which clears the LEDs. Pins 2 and 3 **of the chip** (PD0 = RX, PD1 = TX) must stay free for serial.

Design choices: 8 LEDs for 8 bits (255 = 0b11111111). The two buttons go on **INT0 (PD2)** and **INT1 (PD3)** so no click is missed, each with a 100 nF debounce capacitor. Bits 0–3 on PB0–PB3, bits 4–7 on PD4–PD7, so nothing touches PD0/PD1.

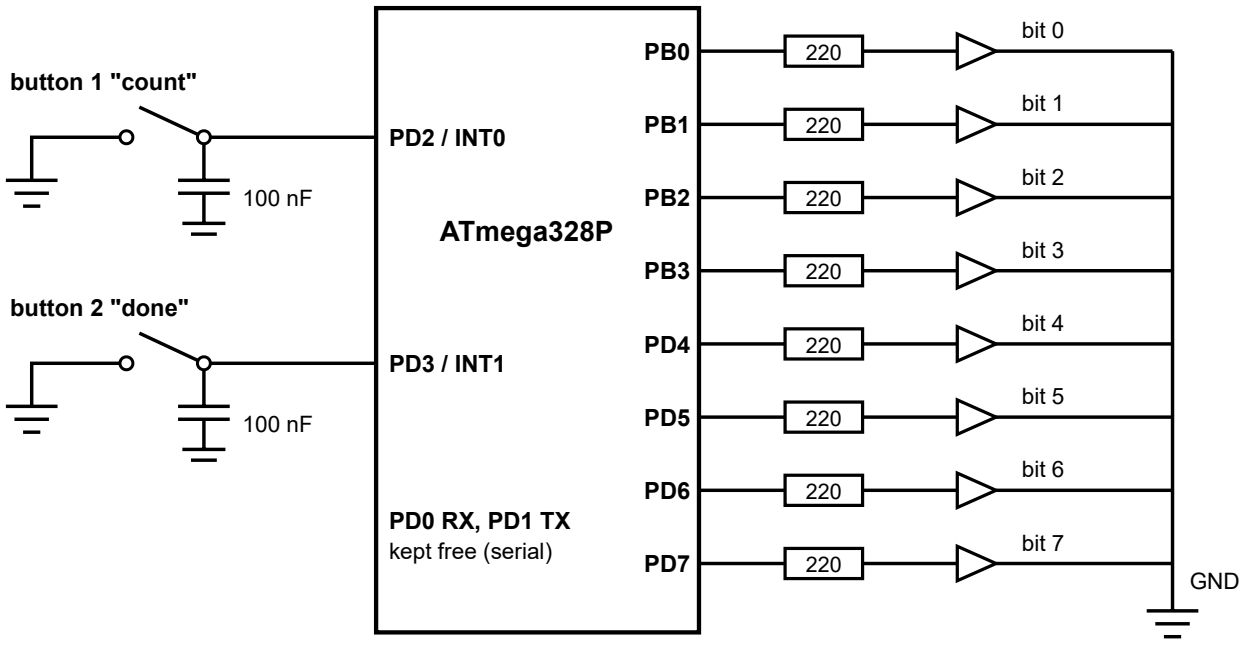


Figure 7. Click counter. Each button goes from its pin to GND with a 100 nF capacitor in parallel; pull-ups are internal. (Supply, reset and crystal not drawn.)

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile unsigned char count = 0;    // changed inside ISRs, so volatile
volatile unsigned char showing = 0;  // 0 = entering a number, 1 = displaying

static void display(unsigned char n)
{
    PORTB = (PORTB & 0xF0) | (n & 0x0F);   // low 4 bits of n on PB0-PB3
    PORTD = (PORTD & 0x0F) | (n & 0xF0);   // high 4 bits of n on PD4-PD7
}

int main(void)
{
    DDRB |= 0x0F;                          // PB0-PB3 outputs
    DDRD |= 0xF0;                          // PD4-PD7 outputs
    DDRD &= ~((1<<PD2) | (1<<PD3));       // PD2, PD3 inputs
    PORTD |= (1<<PD2) | (1<<PD3);         // pull-ups on both buttons
    EICRA = (1<<ISC01) | (1<<ISC11);       // falling edge on INT0 and INT1
    EIMSK = (1<<INT0) | (1<<INT1);        // enable both
    display(0);                            // start with all LEDs off
    sei();
    while (1) { }                          // nothing to do: the ISRs do the work
}

ISR(INT0_vect)                             // button 1: one more click
{
    if (!showing && count < 255) count++;
}

ISR(INT1_vect)                             // button 2: show, or clear
{
    if (!showing) { display(count); showing = 1; }
    else          { display(0); count = 0; showing = 0; }
}
```

Idea	Explanation
`(PORTB & 0xF0) | (n & 0x0F)`	Keep PB4–PB7 as they were, replace PB0–PB3 with the low 4 bits of the number. Example: n = 13 = 0b00001101 lights bits 0, 2, 3.
`(PORTD & 0x0F) | (n & 0xF0)`	Keep PD0–PD3 (so the pull-ups on PD2/PD3 stay on), replace PD4–PD7 with the high 4 bits.
`count < 255`	An `unsigned char` holds 0–255; the 256th click would wrap to 0, so stop at 255.
`volatile`	`count` and `showing` are used by two ISRs; volatile makes every read come from memory.

`while (1) { }`	Everything is event driven, so main just waits. This is fine: interrupts still fire.

This is my own worked answer (no model answer was provided for Exercise 3). Other pin choices are fine if they keep PD0/PD1 free and the buttons on interrupt pins, or use pin change interrupts instead.

BUILDING THESE CIRCUITS

Use the method from Lesson 3, section P (breadboard, schematic, wiring order, fault checklist). The only new part here is the **100 nF capacitor**: one leg in the same column as the button's pin wire (pin 2 / pin 3), the other leg to the GND rail, so it sits in parallel with the button.

Try the theory on the bench: with the System 5/6 wiring, tap the button quickly. Polling (System 5) misses taps; the interrupt (System 6) never does; remove the capacitor and you will sometimes see a double toggle from bounce.

L. What the lecturer explained in the lectures (2020 recordings)

From the four polling and interrupt recordings (folders 09–12 in `Embedded Systems Lectures`). **[EXAM]** marks things he tied to the exam.

> **L1. HOW THE EXAM IS SET [EXAM]**
> "The learning outcomes slide is the most important slide in the entire slide set": the paper is set to check exactly those outcomes. For this lesson they are: compare polling vs interrupts; build a system that does two tasks with polling; explain how interrupt handling works; explain the interrupt registers; build the same system with interrupts.
> **Do not memorise register names or bit positions.** "Even for the exam you get all these names": the practical and the written exam both give you the summarised datasheet. What you must know is *which register to use for what* and how to read the datasheet.

L2. "Seemingly at the same time": multitasking is an illusion

- One CPU can run only **one instruction stream at a time**. It switches between tasks so fast that they look simultaneous, like a juggler who holds only one ball at a time while the others are in the air. This is **multitasking**.
- Truly simultaneous work needs **several CPUs** (a multi-core processor or multi-core microcontroller), each running its own task. This is **multiprocessing**.
- On a PC the **operating system** handles task switching (context switching). A microcontroller has **no OS**, so **multitasking is the programmer's responsibility**. Polling and interrupts are the two ways to do it.
- The purpose: to give the CPU's service to **peripheral devices** while the CPU is busy with another task. Peripherals can be **external** (buttons, sensors) or **internal** (timers, the ADC, the USART).

L3. His polling analogy, and why System 4 was already polling

Reading the newspaper with a baby on the floor. Main task: read the paper. Peripheral: the baby's nappy. With polling you read one article, touch the nappy to check it, read the next article, check again…. If it is wet, you stop, change it, then carry on reading from the next article. **ABS example:** the controller checks wheel 1, 2, 3, 4, then wheel 1 again; if one wheel is slower it releases that brake, then carries on checking.
A student asked how this differs from last week's dual-rate blinker: it doesn't. **System 4 was already polling**: it checked the button once per loop. The idea just wasn't named yet.

L4. Polling's problems, the full list he gave [EXAM]

1. **Missed events**, in two situations: the device changes **too fast** (a quick tap), **or** the checking interval is **too long** (even a normal press is missed if you only check every 5–10 s). His ABS case: while the controller checks wheel 2, wheel 1 slows down and recovers before it is checked again, so the car can slide off course. This is dangerous in a safety-critical system.
2. **No priority:** every task runs in the same fixed order, so an urgent device can't be served first.
3. **Wasted CPU work:** the CPU spends time checking devices that usually don't need anything ("opening the door again and again to see if a visitor has come").

The class experiment (try it yourself): with a 2 s interval, quick taps are often missed and you must hold the button; at 200 ms most presses are caught, but still not all. A shorter interval catches more presses but also changes how fast the LED blinks. Interrupts remove the interval problem altogether.

L5. Two hardware details from the System 5 circuit

- **Why 1 kΩ on the on-board LED but 220 Ω on ours?** The pin 13 LED on the Uno is an **SMD** (surface-mounted device) LED: tiny, needs very little current, so a larger 1 kΩ resistor limits it. Our breadboard LED is a **through-hole** LED (its legs go through holes in the board): bigger, needs more current, so a smaller 220 Ω. He asked this in class.
- **Schematic diagram** = how components are connected, using **standard symbols**. **Wiring diagram** = a simplified pictorial view using **realistic pictures**, showing where parts sit relative to each other.

L6. Interrupts in his words

An interrupt is a **hardware mechanism** (polling is a software technique). The baby **cries** when the nappy is wet, so you just keep reading and react only when it cries. A visitor **knocks** instead of you checking the door. Three gains: **no monitoring work**, **no missed events** however fast or slow they are, and **priority** (if two children cry, you can decide who comes first).

L7. The handling steps exactly as he taught them

1. Execution starts at **address 0**. If interrupts are used, the **interrupt vector table** sits at the start of program memory, and the program **jumps over it** to the code.
2. An interrupt arrives while an instruction is running. The CPU finishes it and saves the **address of the next instruction** (the program counter) on the **stack**.
3. The CPU **turns off interrupts** so that no other interrupt breaks in while this one is handled.
4. It jumps to the **vector table**, which holds the **address of each ISR**, then jumps to the right ISR and runs it. You, the programmer, write what the ISR does.
5. At the end of the ISR it **turns interrupts on again**, takes the saved address off the stack, and **resumes** exactly where it stopped.

His analogies: you put a **bookmark** in page 5 (the stack), go and change the nappy (the ISR), then open the book at the bookmark (the return). While changing the older brother's nappy you **stop the younger one from crying** (interrupts disabled); afterwards you let him cry again (interrupts re-enabled).

> **KEEP ISRS SHORT [EXAM REASONING]**
> While an ISR runs, all other interrupts are disabled. A long ISR therefore (1) delays the main task, and (2) can make other events go **unnoticed**. His example: a long ADC ISR is still running when the timer overflows, so the overflow interrupt is missed. Writing short ISRs is **the programmer's responsibility**; there is no OS to protect you.

L8. Priority and the vector table

Each interrupt source has its **own ISR** and its own vector-table entry. **Lower vector address = higher priority**: if INT0 and the ADC interrupt arrive together, INT0 runs first. INT0 has a higher priority than INT1. Timer overflow happens when a timer counts past its maximum (255 for an 8-bit timer) back to 0.

L9. The stack (and an aside on recursion)

The stack is **last in, first out**. When main calls A, A calls B and B calls C, the return addresses are stacked; they come back off in the order C → B → A → main. Each call also stores its local variables there. **Deep recursion** keeps adding to the stack and can **run out of memory and crash** the system; attackers have exploited this. Recursion is still the natural solution to some problems (the classic *Towers of Hanoi*), but for performance and safety he prefers iteration on small machines.

L10. Rules of an ISR in C, and why [EXAM]

- Written as `ISR(vector_name) { ... }`. Books also call it an **interrupt handler**.
- **No return value** and **no parameters** except the vector. **Why?** It is called automatically by the hardware at a moment nobody knows when the code is written or compiled. There is no calling statement, so there is nowhere to pass arguments from and nowhere to return a value to. ("If you are asked why ISRs look like functions but have no return type and no arguments, can you explain?")
- **To pass data in or out, use global variables declared `volatile`. Why volatile?** The compiler sees no call to the ISR, so it assumes the ISR never runs and that a variable changed only inside it never changes. With optimisation on, it may then replace the variable with a **literal constant** (e.g. always 5), and main never sees the change. `volatile` means "this can change at any time, don't optimise it away".
- **Literal constant** = a plain number in the code (`5`), with no memory reserved. A `const int y = 5;` still occupies memory.
- **All interrupts are disabled by default**, and most peripherals are off by default, **to save power**. `sei()` sets the **I bit (bit 7) of SREG**, the status register; `cli()` clears it.
- **[EXAM]** `sei()`, `cli()`, `ISR()` and the vector names are defined in `avr/interrupt.h`. "One time" an exam gave interrupt code with blanks to fill, and the `#include <avr/interrupt.h>` line was missing. **No student noticed it**, although without it the program does not compile.

L11. External interrupt details

- The ATmega328P has only **two** external interrupt pins: **INT0 = PD2 = Arduino pin 2**, **INT1 = PD3 = Arduino pin 3**. More need a different chip, or pin change interrupts.
- **Low level** means the pin must be **held LOW until the current instruction finishes**. **Falling edge** = "going low" (the moment between HIGH and LOW). **Rising edge** = "going high". **Logic change** = either.
- INT0 and INT1 can use **different** trigger choices (e.g. INT0 on any change, INT1 on rising edge).
- EICRA only chooses **when**; EIMSK **enables**. You need both, plus `sei()`. In System 6 only INT0 is enabled, because nothing is connected to INT1.
- **How to read his register slides:** bits with names are explained on that slide; "n/a" bits are explained later in the set; "–" bits are not covered in that slide set. If you are curious, read the datasheet.

L12. Capacitors, as he taught them

Point	Detail
What it is	Two metal plates with an **insulator** (ceramic, mica, polyester…) between them. It stores charge **temporarily**: the charge slowly leaks away (the same reason DRAM must be refreshed).
Unit	**Farad** (after Michael Faraday). It is huge, so we use µF, nF, pF: 1000 pF = 1 nF, 1000 nF = 1 µF.
Typical use	**Smoothing a power supply**: placed across + and GND, it charges, then discharges into any dip, so the device sees a steady voltage.
Polarised (electrolytic)	Has + and − legs that must go the right way. Value and max voltage are printed (e.g. 10 µF 50 V).
Non-polarised (ceramic)	Either way round. **Code "104"** = 10 followed by 4 zeros pF = 100,000 pF = **100 nF = 0.1 µF** (the one in your kit). Voltage code e.g. **2E = 250 V**. Tolerance letter: **K = ±10%**, J = ±5%. Cheap ones may show only the value.

L13. Bounce and debouncing in his words

The contacts of a mechanical switch don't close once and stay closed: due to their **momentum they bounce** several times in a few milliseconds before settling. This is the nature of switches, not a fault. The microcontroller sees several HIGH–LOW changes, so with interrupts enabled **one press can interrupt the CPU several times** and the LED toggles several times. **Hardware fix:** 100 nF across the switch. When the switch is open, the capacitor is **charged through the internal pull-up**; during bouncing it **discharges and fills the gaps**, giving one clean change, "like a reservoir". **Software fix:** he set this as **homework** ("you know coding, find it out"). One answer is the edge-detect-plus-short-delay code in section 2 of these notes.

L14. Pin change interrupts: extra points

- Why they exist: INT0/INT1 give only two interrupt pins, but pin change interrupts make **virtually every I/O pin** interrupt-capable.
- Difference from external interrupts: **only one trigger choice, "logic change"** (equivalent to the INT "logic change" option), but on **any pin**.
- Two steps: **enable the port** with PCICR (PCIE0 = port B, PCIE1 = port C, PCIE2 = port D), **then the pin** with PCMSK0/1/2.
- **Why is bit 7 of PCMSK1 unused?** PCMSK1 is port C, and port C has only **7 pins** (PC0–PC6), so there is no pin for bit 7. (A student asked this.)
- **One pin, several labels:** Arduino **2** = **PD2** (as I/O) = **INT0** (as external interrupt) = **PCINT18** (as pin change interrupt). All mean the same pin. **Use the label that matches how you use it**, so the reader is not misled (writing INT0 in pin change code suggests an external interrupt).
- The ISR runs on **press and release**, so it must test the pin. "Currently LOW and an interrupt happened" means it was HIGH before: a **falling edge**, a press. "Currently HIGH" means a release.
- `DDRD &= ~(1<<2);` is not strictly needed (pins are inputs after reset) but is **good practice** for readability, at the cost of one clock cycle.
- **Arduino has no official pin change interrupt support**, so in Arduino you only have the two INT pins. A **third-party library, PinChangeInt**, exists but is not official.

L15. Arduino's attachInterrupt

`digitalPinToInterrupt(2)` converts Arduino pin 2 to interrupt number 0 (INT0), so Arduino users never need to know interrupt numbers. `attachInterrupt(num, toggle, FALLING)` links a normal function to it; behind the scenes the real ISR calls your function. Modes: **LOW, CHANGE, RISING, FALLING**. "Most Arduino users don't know you can do interrupts at all."

L16. Code style and addresses

- Binary strings, hex (`0x21`), shifts (`(1<<5)|(1<<0)`) and labels (`PCINT18`) give **exactly the same machine code**: no difference in speed or size, because the compiler converts them. Shifts and labels matter most on 32-bit chips, where counting 32 bits is error-prone.
- Register names are **labels for memory addresses**. `avr/io.h` is common to all AVR chips; the chip-specific `iom328p.h` (added when you select the device) gives the real addresses. Each I/O register has an **I/O address** (relative to the start of I/O memory) and a **data-memory address**.
- Size of the System 5 firmware: **C ≈ 180 bytes, Arduino > 1100 bytes**.

L17. Assignment hints he gave

- Exercise 2 Q1 in assembly: use `SBIC PIND, 2` (skip the next instruction if the bit is clear, i.e. the button is pressed) to skip one call of a **200 ms delay**, and call it **twice** for 400 ms. Writing one 200 ms routine and calling it twice is better than two separate delay loops.
- Toggle in assembly with `EOR` (exclusive OR). Move a port register into a working register with `IN` and back with `OUT`.
- He refused to hand out full answers: "otherwise you become good typists, not good programmers". **Do every practical yourself**, try different polling intervals, and bring your questions. Exercise 3 (the click counter) belongs to this lesson.

9. Common mistakes

Mistake	Result / fix
Forgot `sei()`	No interrupt ever runs. The local enable alone is not enough.
Forgot `#include <avr/interrupt.h>`	`ISR` and `sei` are unknown, so compile error.
Wrong vector name (e.g. `INT0vect`)	Compiles with a warning but the ISR is never used. Copy the name from the exam data sheet.
Button on a pin other than PD2/PD3 with INT0/INT1	Never triggers. Use pins 2/3, or a pin change interrupt.
PCINT without checking the pin in the ISR	Runs on press and release, so it toggles twice.
Setting PCMSK but not PCICR (or the other way)	Both are needed: PCICR enables the port, PCMSK picks the pins.
Long delays inside an ISR	Blocks everything else. Set a flag, do the work in main.
Shared variable not `volatile`	Main may never see the ISR's change.
No debounce capacitor	One press can count as several.

Cheat sheet: what is NOT on the exam sheets

Concepts	
Polling	CPU checks devices repeatedly
Problems	1. misses fast/short events 2. no priority
Interrupt	device asks; CPU suspends, runs ISR, resumes
Steps	finish instruction → push return address → vector → ISR → pop → resume
Priority	lower vector address = higher priority (INT0 highest)
Enables	local bit AND global I bit (`sei()`)
Debounce	100 nF from pin to GND

Setup recipes	

INT0 falling	`EICRA |= (1<<ISC01);`
	`EICRA &= ~(1<<ISC00);`
	`EIMSK |= (1<<INT0); sei();`
	`ISR(INT0_vect)`

PCINT on PD2	`PCICR |= (1<<PCIE2);`
	`PCMSK2 |= (1<<PCINT18); sei();`
	`ISR(PCINT2_vect)` + check `PIND`

Pins	INT0 = PD2 = pin 2, INT1 = PD3 = pin 3
PCINT groups	B: 0–7 (PCMSK0) C: 8–14 (PCMSK1) D: 16–23 (PCMSK2)